

SERVICIO NACIONAL DE APRENDIZAJE — SENA  
Centro de Servicio y Gestión Empresarial

# MANUAL TÉCNICO

*AutoPrime — Plataforma web para un atelier automotriz*

|                                       |                                                                                                                                                                                                                                                                                                                                          |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Aprendiz</b>                       | José Matías Agudelo Bolívar                                                                                                                                                                                                                                                                                                              |
| <b>Documento de identidad</b>         | 1038928023                                                                                                                                                                                                                                                                                                                               |
| <b>Ficha de caracterización</b>       | 3406211                                                                                                                                                                                                                                                                                                                                  |
| <b>Programa de formación</b>          | Tecnólogo en Análisis y Desarrollo de Software (código 228118)                                                                                                                                                                                                                                                                           |
| <b>Centro / Regional</b>              | Centro de Servicios y Gestión Empresarial — Coordinación de<br>Teleinformática, SENA Regional Antioquia                                                                                                                                                                                                                                  |
| <b>Competencia</b>                    | 38367 — Estructurar propuesta técnica de servicio de TI según requisitos<br>técnicos y normativa<br>38356 — Implementar la solución de software de acuerdo con los requisitos<br>de operación y modelos de referencia                                                                                                                    |
| <b>Resultado de Aprendizaje (RAP)</b> | 593060-01 — Definir especificaciones técnicas del software de acuerdo con<br>las características del software a construir<br>593109-03 — Documentar el proceso de implantación de software<br>siguiendo estándares de calidad<br>593112-04 — Implantar el software de acuerdo con los niveles de servicio<br>establecidos con el cliente |
| <b>Instructor</b>                     | César Augusto Moreno Mena                                                                                                                                                                                                                                                                                                                |
| <b>Fecha de entrega</b>               | 24 de septiembre de 2026                                                                                                                                                                                                                                                                                                                 |

Repositorio: <https://github.com/matias2421/autoprim>  
API desplegada: <https://autoprim-api-z9b6.onrender.com>

## Tabla de contenido

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>1. Introducción y descripción general</b>            | <b>4</b>  |
| 1.1 El problema que resuelve                            | 4         |
| 1.2 Contexto                                            | 4         |
| 1.3 Usuarios y actores                                  | 4         |
| <b>2. Objetivos</b>                                     | <b>5</b>  |
| 2.1 Objetivo general                                    | 5         |
| 2.2 Objetivos específicos                               | 5         |
| <b>3. Alcance del proyecto</b>                          | <b>5</b>  |
| 3.1 Funcionalidades incluidas                           | 5         |
| 3.2 Funcionalidades explícitamente excluidas            | 6         |
| <b>4. Arquitectura de la solución</b>                   | <b>7</b>  |
| 4.1 Diagrama de arquitectura                            | 7         |
| 4.2 Stack tecnológico                                   | 8         |
| 4.3 Organización del código                             | 8         |
| 4.4 Comparativa: FastAPI frente a Django REST Framework | 9         |
| <b>5. Modelo de datos</b>                               | <b>10</b> |
| 5.1 Diagrama entidad-relación                           | 10        |
| 5.2 Decisiones del modelo que conviene conocer          | 10        |
| 5.3 Sobre la tabla de permisos                          | 11        |
| 5.4 Diccionario de datos                                | 11        |
| <b>6. Diseño de la solución</b>                         | <b>16</b> |
| 6.1 Historias de usuario principales                    | 16        |
| 6.2 Flujo de una venta, de principio a fin              | 16        |
| <b>7. Manual de instalación y configuración</b>         | <b>18</b> |
| 7.1 Requisitos previos                                  | 18        |
| 7.2 Instalación del backend                             | 18        |
| 7.3 Variables de entorno                                | 18        |
| 7.4 Preparar la base de datos                           | 19        |
| 7.5 Arrancar                                            | 19        |

|                                                        |           |
|--------------------------------------------------------|-----------|
| 7.6 Despliegue en producción . . . . .                 | 20        |
| <b>8. Documentación técnica de la API . . . . .</b>    | <b>21</b> |
| 8.1 Formato de las respuestas . . . . .                | 23        |
| 8.2 Ejemplo: iniciar sesión . . . . .                  | 23        |
| 8.3 Ejemplo: registrar una venta . . . . .             | 24        |
| 8.4 Ejemplo: un error de validación . . . . .          | 25        |
| <b>9. Manual de usuario . . . . .</b>                  | <b>26</b> |
| 9.1 Entrar al sistema . . . . .                        | 26        |
| 9.2 El tablero del administrador . . . . .             | 26        |
| 9.3 Registrar una venta . . . . .                      | 27        |
| 9.4 Emitir y descargar una factura . . . . .           | 27        |
| 9.5 Atención al cliente (PQR) . . . . .                | 28        |
| 9.6 El asistente . . . . .                             | 29        |
| 9.7 Lo que ve un cliente . . . . .                     | 30        |
| <b>10. Pruebas realizadas . . . . .</b>                | <b>31</b> |
| 10.1 Pruebas automáticas . . . . .                     | 31        |
| 10.2 La colección de Postman . . . . .                 | 31        |
| 10.3 Pruebas de seguridad . . . . .                    | 32        |
| 10.4 Defectos encontrados y corregidos . . . . .       | 34        |
| <b>11. Conclusiones y recomendaciones . . . . .</b>    | <b>35</b> |
| 11.1 Conclusiones . . . . .                            | 35        |
| 11.2 Recomendaciones para una futura versión . . . . . | 35        |
| <b>12. Anexos . . . . .</b>                            | <b>36</b> |
| 12.1 Enlaces . . . . .                                 | 36        |
| 12.2 Cuentas de demostración . . . . .                 | 36        |
| 12.3 Dependencias del backend . . . . .                | 37        |
| 12.4 Dependencias del frontend . . . . .               | 38        |
| 12.5 Otras evidencias entregadas . . . . .             | 38        |
| 12.6 Cómo se generó este manual . . . . .              | 38        |

## 1. Introducción y descripción general

### 1.1 El problema que resuelve

Un atelier automotriz —un concesionario de vehículos de alta gama con taller propio— mueve pocas operaciones al mes, pero cada una vale mucho y deja rastro en varios sitios a la vez: el vehículo sale del catálogo, se emite una factura, queda un saldo por cobrar y casi siempre una cita de entrega o de taller.

Cuando ese rastro se lleva en una hoja de cálculo y en el correo aparecen tres problemas, y son los que ataca este proyecto. El primero es que **nadie sabe en tiempo real cómo va el negocio**: para responder «cuánto llevamos este mes» o «cuánto hay pendiente de cobro» hay que sentarse a sumar. El segundo es que **el dato no tiene un dueño claro**: el mismo número vive en la hoja de un vendedor y en el correo de otro, y cuando no coinciden no hay forma de saber cuál es el bueno. El tercero es que **el cliente queda fuera**: para conocer el estado de su compra, pedir una cita o reclamar, tiene que llamar y esperar a que alguien lo atienda.

AutoPrime resuelve los tres con la misma decisión de fondo: una sola base de datos, una API que es lo único que la toca, y tres vistas distintas sobre esa misma información según quién entre.

### 1.2 Contexto

El proyecto se desarrolla como proyecto formativo del programa de Análisis y Desarrollo de Software, ficha 3406211. El negocio que se modela es ficticio pero coherente: un atelier del Eje Cafetero que vende vehículos de alta gama personalizados y presta servicios de peritaje, prueba de manejo y taller certificado.

Las reglas que aplica son las colombianas. El IVA se calcula al 19 %, los importes van en pesos y se redondean una sola vez, los teléfonos se validan como celulares de diez dígitos que empiezan por 3, las fechas se interpretan en la zona horaria de Bogotá y la agenda respeta los festivos nacionales.

La zona horaria merece una línea aparte porque costó un fallo real: el servidor de producción trabaja en UTC, y sin fijar la zona el informe del día salía vacío a partir de las siete de la tarde, porque para el servidor ya era mañana. Toda fecha del sistema pasa hoy por un único módulo que resuelve esa cuestión.

### 1.3 Usuarios y actores

El sistema reconoce tres roles, y la diferencia entre ellos no es cosmética: cada uno ve pantallas distintas y, sobre todo, la API le responde cosas distintas aunque pida exactamente la misma ruta.

| Actor                            | Qué puede hacer                                                                                                  | Qué no puede hacer                                                                                    |
|----------------------------------|------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Visitante</b><br>(sin cuenta) | Ver el catálogo y la ficha de cada vehículo, consultar los servicios, preguntar al asistente y crear una cuenta. | Nada que toque datos de nadie.                                                                        |
| <b>Cliente</b>                   | Ver sus compras, sus facturas, sus citas y sus PQR. Agendar, radicar reclamos y descargar sus propias facturas.  | Ver operaciones de otros clientes, registrar ventas, emitir facturas o entrar al tablero del negocio. |
| <b>Empleado</b>                  | Registrar ventas, emitir y anular facturas, gestionar la agenda y responder PQR.                                 | Administrar usuarios ni consultar los informes del negocio.                                           |
| <b>Administrador</b>             | Todo lo del empleado, más la gestión de usuarios y del catálogo y el tablero con los informes en PDF y Excel.    | —                                                                                                     |

Tabla 1. Actores del sistema y alcance de cada uno.

## 2. Objetivos

### 2.1 Objetivo general

Desarrollar y desplegar una aplicación web que permita a un atelier automotriz gestionar en un solo lugar su catálogo, sus ventas, su facturación, su agenda y la atención a sus clientes, con acceso diferenciado por rol y con la información del negocio disponible en el momento en que se necesita.

### 2.2 Objetivos específicos

1. **Modelar y construir la base de datos relacional** que sostiene el negocio, con la integridad garantizada por claves ajenas y no por la disciplina de quien escribe.
2. **Construir una API REST con FastAPI** que sea el único camino hacia los datos, que valide todo lo que entra y que documente sola lo que ofrece.
3. **Implementar autenticación y autorización por rol** con JWT y contraseñas cifradas con bcrypt, de modo que el permiso lo decida siempre el servidor y nunca la interfaz.
4. **Desarrollar la interfaz en React** consumiendo esa API, con validación en vivo en los formularios y tres vistas separadas según quién haya entrado.
5. **Generar los documentos del negocio** —facturas e informes en PDF, e informes en Excel— desde el propio servidor, con los cálculos hechos una sola vez y en un solo sitio.
6. **Desplegar el sistema en internet y dejarlo comprobable:** pruebas automáticas, una batería de peticiones sobre la API y medidas de seguridad verificables sobre el servidor en producción.

## 3. Alcance del proyecto

Decir qué hace un sistema solo sirve si también se dice qué no hace. Lo segundo se documenta aquí con el mismo detalle que lo primero: son decisiones tomadas, no cosas que se olvidaron.

### 3.1 Funcionalidades incluidas

| Módulo           | Qué incluye                                                                                                                                                |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Catálogo         | Alta, edición, baja y consulta de vehículos y de servicios, con ficha técnica, familia, estado y galería. Consulta pública.                                |
| Cuentas y acceso | Registro de clientes, inicio de sesión con JWT, recuperación de contraseña por correo, gestión de usuarios y baja lógica.                                  |
| Agenda           | Consulta de franjas libres, reserva por servicio y vehículo, reprogramación y cambio de estado. Respeto festivos.                                          |
| Ventas           | Registro con varias líneas, cálculo de subtotal, descuento e IVA en el servidor, estados (pendiente, pagada, anulada) y consulta con filtros y paginación. |
| Facturación      | Emisión a partir de una venta, numeración propia, anulación sin borrado y descarga en PDF con importe en letras y código de verificación.                  |
| Informes         | Tablero con cifras del periodo y comparación con el anterior, gráficas de ingresos por día y de lo más vendido, y descarga del informe en PDF y en Excel.  |
| PQR              | Radicación por el cliente con número de radicado, respuesta y cambio de estado por parte del atelier.                                                      |
| Asistente        | Chat con un modelo de lenguaje que responde sobre el catálogo, los servicios y la agenda, con la conversación guardada.                                    |
| Seguridad        | Cabeceras de seguridad, límite de intentos en el acceso y en la recuperación, consultas parametrizadas y escapes en los documentos generados.              |

Tabla 2. Alcance funcional implementado.

### 3.2 Funcionalidades explícitamente excluidas

| Queda fuera                           | Por qué                                                                                                                                                                          |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pasarela de pago en línea             | Exige un contrato con una entidad recaudadora y tratar datos de tarjeta, que es justo lo que un proyecto formativo no debe manipular. La venta registra el cobro; no lo ejecuta. |
| Facturación electrónica ante la DIAN  | Requiere habilitación como facturador y firma digital. El PDF reproduce la estructura de una factura, pero no es un documento fiscal válido, y el propio documento lo advierte.  |
| Aplicación móvil nativa               | La interfaz es adaptable y funciona en el navegador del móvil. Una aplicación nativa sería otro proyecto.                                                                        |
| Inventario por unidades y proveedores | El catálogo lleva el estado de cada vehículo (disponible o vendido), no un almacén con entradas, salidas y compras.                                                              |
| Nómina, contabilidad y cartera        | El sistema sabe qué está por cobrar, pero no lleva libros contables ni gestiona pagos a proveedores.                                                                             |
| Mensajería entre personas             | El asistente habla con un modelo de lenguaje. No hay chat entre cliente y vendedor; para eso están las PQR y WhatsApp.                                                           |

Tabla 3. Fuera de alcance, y el motivo.

## 4. Arquitectura de la solución

La arquitectura es de tres capas con una separación estricta: el navegador nunca habla con la base de datos. Todo pasa por la API, y eso no es una formalidad — es lo que permite que la misma regla de negocio valga igual para la interfaz, para una petición hecha desde Postman y para un script.

### 4.1 Diagrama de arquitectura

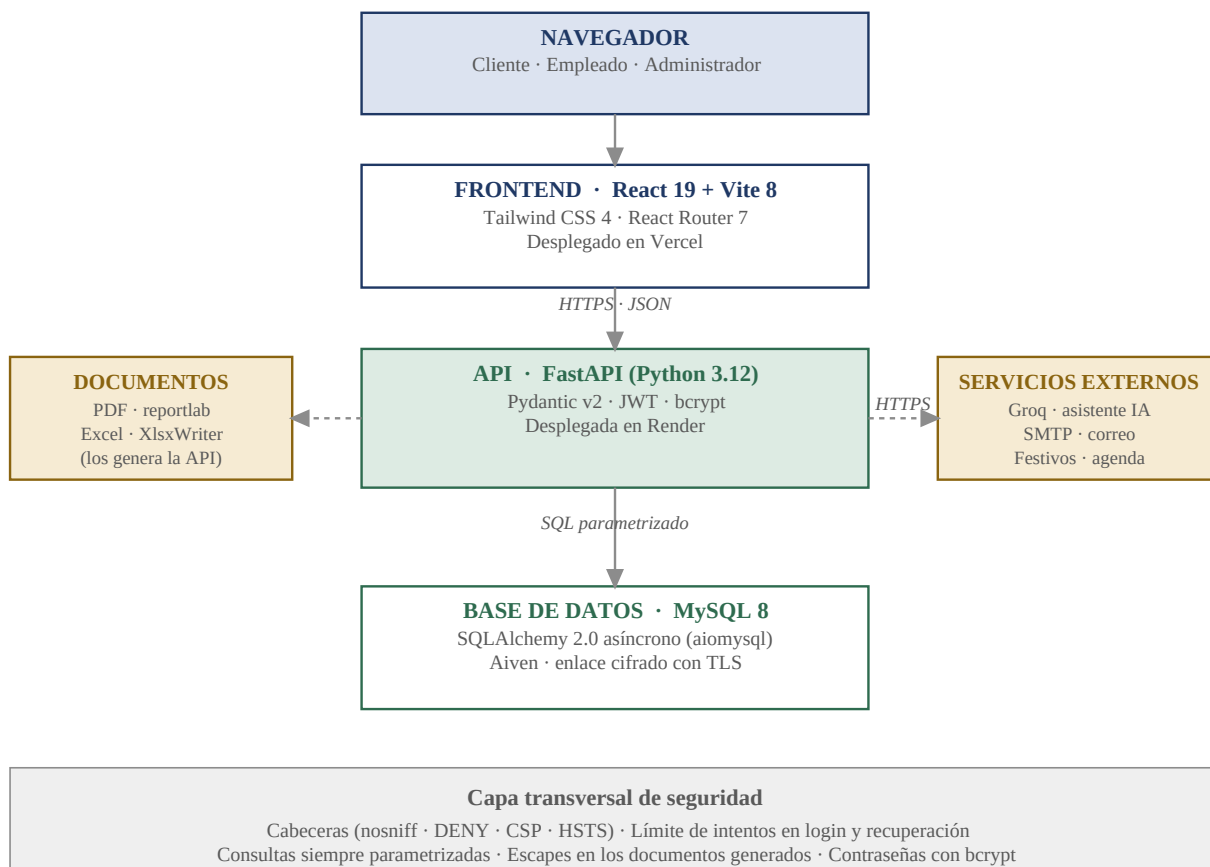


Figura 1. Capas del sistema, servicios externos y la capa transversal de seguridad.

## 4.2 Stack tecnológico

| Capa               | Tecnología                                          | Por qué esta y no otra                                                                                                                                                              |
|--------------------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend           | React 19 · Vite 8 · Tailwind CSS 4 · React Router 7 | Vite da recarga instantánea en desarrollo y un paquete pequeño en producción. Tailwind mantiene el estilo junto al componente, sin hojas sueltas que nadie sabe si siguen usándose. |
| Backend            | Python 3.12 · FastAPI · Pydantic v2 · Uvicorn       | FastAPI valida con Pydantic y publica su propia documentación OpenAPI: la especificación no puede quedarse atrás del código porque sale del código.                                 |
| Acceso a datos     | SQLAlchemy 2.0 asíncrono · aiomysql                 | El ORM parametriza todas las consultas, que es la defensa real contra la inyección SQL. En asíncrono, una consulta lenta no bloquea al resto de peticiones.                         |
| Base de datos      | MySQL 8 (Aiven en producción, XAMPP en local)       | Relacional porque el negocio lo es: una factura sin su venta no significa nada, y eso lo garantiza una clave ajena.                                                                 |
| Documentos         | reportlab (PDF) · XlsxWriter (Excel)                | Son Python puro y no arrastran librerías del sistema. Un generador que dependiera de Chrome o de cairo funcionaría en el portátil y fallaría en el servidor.                        |
| Servicios externos | Groq · SMTP · calendario de festivos                | El asistente usa un modelo de lenguaje alojado; el correo sale por SMTP; los festivos se consultan para no ofrecer citas en días no hábiles.                                        |
| Despliegue         | Render (API) · Vercel (frontend)                    | Los dos despliegan desde el repositorio en cada push. La API necesita un proceso vivo; el frontend son archivos estáticos.                                                          |

Tabla 4. Stack tecnológico y criterio de elección.

## 4.3 Organización del código

El backend está dividido por responsabilidad, no por entidad. Cada carpeta de `backend/app` hace una sola cosa:

- **core** — configuración, conexión a la base, seguridad, cabeceras y el módulo de fechas.
- **models** — las tablas, declaradas con SQLAlchemy.
- **schemas** — lo que entra y lo que sale, validado con Pydantic. Separado de los modelos a propósito: así una columna interna como `password_hash` no puede escaparse en una respuesta por descuido.
- **crud** — las operaciones sobre la base y las reglas de negocio.
- **routers** — las rutas HTTP, que solo orquestan.
- **reportes** — la generación de PDF y Excel, con sus escapes.

El frontend sigue la misma idea: `pages` para las pantallas, `components` para lo reutilizable, `hooks` para la lógica compartida, `context` para el estado de sesión y `api` para el único cliente HTTP que habla con el backend.

| Parte                   | Archivos | Líneas |
|-------------------------|----------|--------|
| Backend (backend/app)   | 57       | 8.352  |
| Frontend (frontend/src) | 69       | 11.182 |
| Pruebas (backend/tests) | 8        | 1.904  |

Tabla 5. Tamaño del proyecto.

## 4.4 Comparativa: FastAPI frente a Django REST Framework

Las dos son opciones razonables para construir esta API en Python, y la elección no es de gustos: cambian cosas concretas del proyecto. Lo que sigue no es una comparación general de los dos marcos, sino lo que habría pasado **en AutoPrime** con cada uno.

| Aspecto                  | FastAPI (lo elegido)                                                                                                                                                                                                                          | Django REST Framework                                                                                                                                                                      |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Validación de datos      | Pydantic v2. El tipo de Python es la validación: <code>Annotated[int, Path(ge=1)]</code> valida el identificador de la ruta, y <code>Literal</code> cierra los vocabularios (estados, tipos de documento) sin escribir una sola comprobación. | Serializadores propios. Hay que declarar el campo en el modelo y otra vez en el serializador. Es más ceremonia, pero deja la validación y la representación explícitas en un solo archivo. |
| Documentación            | El OpenAPI sale del código, siempre al día. <b>De ahí se genera la colección de Postman de este proyecto</b> , y eso solo es fiable porque la especificación no puede quedarse atrás.                                                         | Necesita drf-spectacular o similar, y hay que anotar las vistas para que el esquema salga bien. Una ruta nueva puede quedar sin documentar sin que nada avise.                             |
| Asincronía               | ASGI nativo. Este proyecto tiene 158 funciones asíncronas, habla con MySQL por <code>aiomysql</code> y con el modelo de lenguaje por <code>httpx</code> . <b>Una respuesta del asistente que tarda tres segundos no bloquea a nadie más.</b>  | WSGI de origen; su soporte asíncrono existe pero es parcial y el ORM sigue siendo síncrono. La llamada al asistente ocuparía un proceso entero mientras espera.                            |
| Acceso a datos           | No trae ORM: hubo que elegir e integrar SQLAlchemy 2.0. Más trabajo inicial y más decisiones que tomar.                                                                                                                                       | <b>Aquí gana.</b> El ORM viene integrado, con migraciones y panel de administración. Este proyecto crea el esquema con un script y aún no tiene migraciones: con Django ya las tendría.    |
| Autenticación y permisos | Se construyeron a mano: JWT, <code>bcrypt</code> y dependencias que comprueban el rol. Unas 200 líneas, pero hacen exactamente lo que el negocio necesita y nada más.                                                                         | Trae usuarios, grupos y permisos hechos. Habría ahorrado ese trabajo a cambio de adaptarse a su modelo de usuario.                                                                         |
| Curva y tamaño           | Se parte de casi nada y se añade lo que haga falta. El proyecto usa 19 dependencias en total.                                                                                                                                                 | Trae muchas piezas resueltas de fábrica; a cambio hay que aprender sus convenciones y cargar con lo que no se use.                                                                         |

Tabla 6. Los dos marcos, aplicados a las decisiones reales de AutoPrime.

### Conclusión de la comparativa

**FastAPI fue la elección correcta para este proyecto, y por dos motivos concretos.** El primero es el asistente: una llamada a un modelo de lenguaje tarda segundos, y sin asincronía nativa cada pregunta dejaría un proceso ocupado esperando. El segundo es que la batería de pruebas de la API **se genera del OpenAPI**; con una especificación que hay que mantener aparte, esa automatización no se sostiene.

Pero conviene no barrer para casa: **Django REST Framework habría dado gratis dos cosas que aquí faltan.** Las migraciones de esquema —que en el apartado 11.2 aparecen como recomendación pendiente— y un panel de administración para el catálogo, que hoy se gestiona por la API. En un proyecto con menos necesidad de asincronía y más de administrar datos rápido, la balanza se inclinaría al otro lado.

## 5. Modelo de datos

El esquema tiene **13 tablas**. La integridad no depende de que el código se acuerde de comprobar: las claves ajenas impiden que exista una factura sin venta, una línea sin factura o un mensaje sin conversación.

### 5.1 Diagrama entidad-relación

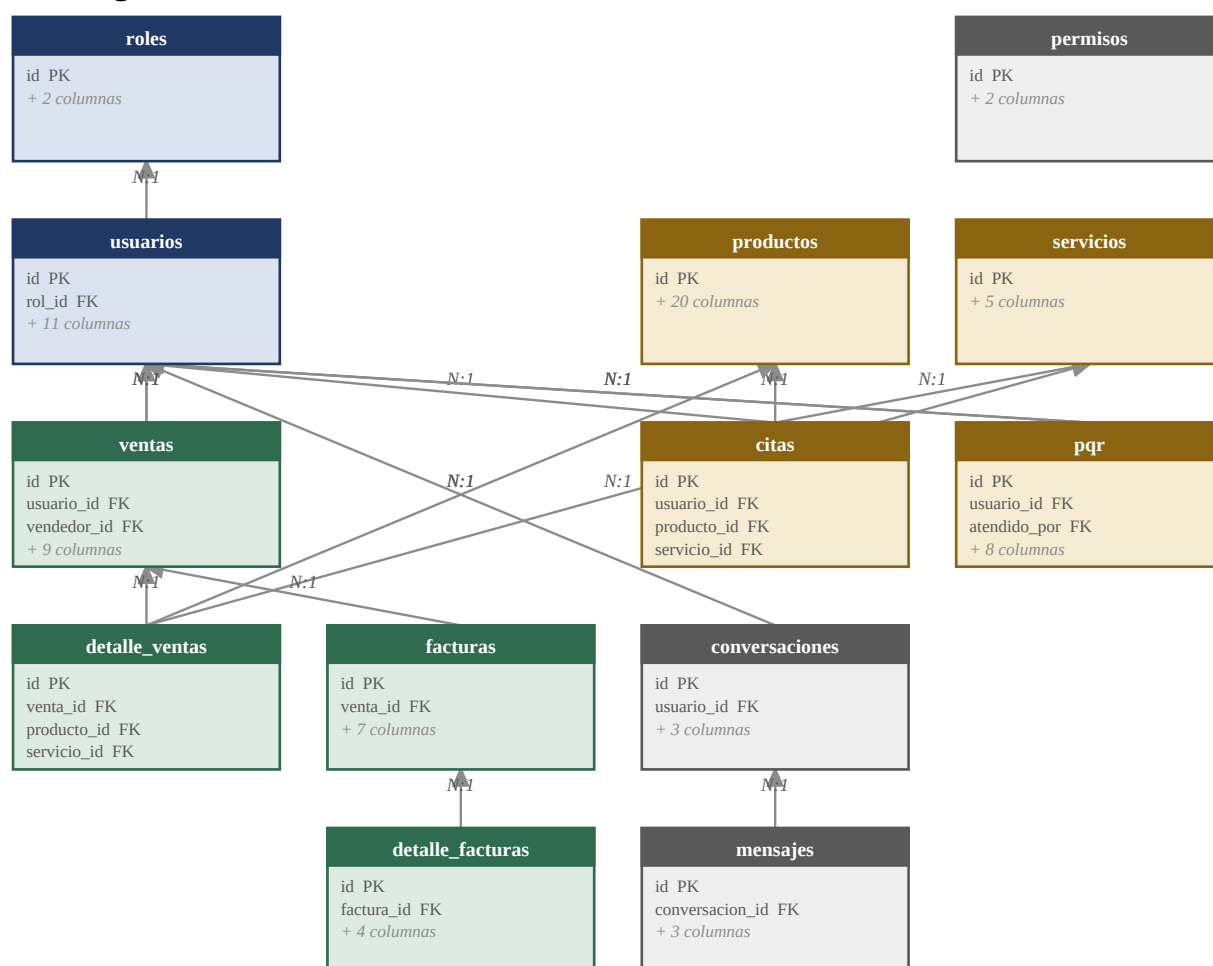


Figura 2. Entidades y claves ajenas. Cada flecha va del lado «muchos» al lado «uno». Se dibuja leyendo el esquema, así que no puede discrepar de él.

### 5.2 Decisiones del modelo que conviene conocer

- **Las líneas guardan copia de la descripción y del precio.** Si mañana sube el precio de un vehículo, una venta de hace tres meses tiene que seguir valiendo lo que valió. Por eso **detalle\_ventas** no se limita a apuntar al producto: copia lo que se vendió y a cuánto.
- **El estado del catálogo describe el presente, no el pasado.** Que un vehículo se vendiera hace cuarenta días no obliga a que hoy siga marcado como vendido: el atelier repone. Por eso el estado vive en **productos** y el histórico en las ventas.
- **Una factura anulada no se borra.** Cambia de estado. Un documento fiscal que desaparece deja un hueco en la numeración, y un hueco en la numeración es exactamente lo que no debe pasar.

- **Los importes van en DECIMAL(14,2), nunca en coma flotante.** Un número en coma flotante no puede representar 0,1 con exactitud, y en dinero eso acaba en céntimos que no cuadran.

### 5.3 Sobre la tabla de permisos

El modelo declara una tabla `permisos` con un catálogo de acciones nombradas, heredada de una etapa anterior del proyecto. Conviene ser explícito: **la autorización real no la resuelve esa tabla**, sino el nombre del rol del usuario, que es lo que comprueban las dependencias de FastAPI en cada ruta protegida. La tabla se conserva porque describe bien el vocabulario de acciones del sistema y porque una futura versión con permisos finos partiría de ahí, pero hoy no interviene en ninguna decisión.

### 5.4 Diccionario de datos

Se lee directamente de la declaración de SQLAlchemy, de modo que coincide con el esquema por construcción y no por revisión.

*permisos — Catálogo de acciones nombradas. Ver la nota del apartado 7.3.*

| Campo       | Tipo         | Obligatorio | Notas          |
|-------------|--------------|-------------|----------------|
| id          | INT          | Sí          | Clave primaria |
| nombre      | VARCHAR(60)  | Sí          | Único          |
| descripcion | VARCHAR(150) | Sí          | —              |

**productos** — *El catálogo de vehículos, con su ficha técnica y su estado.*

| Campo       | Tipo          | Obligatorio | Notas          |
|-------------|---------------|-------------|----------------|
| id          | INT           | Sí          | Clave primaria |
| slug        | VARCHAR(60)   | Sí          | Único          |
| marca       | VARCHAR(40)   | Sí          | —              |
| modelo      | VARCHAR(60)   | Sí          | —              |
| familia     | VARCHAR(9)    | Sí          | —              |
| base        | VARCHAR(60)   | Sí          | —              |
| lema        | VARCHAR(120)  | Sí          | —              |
| descripcion | VARCHAR(1000) | Sí          | —              |
| imagen      | VARCHAR(120)  | Sí          | —              |
| anio        | SMALLINT      | Sí          | —              |
| kilometraje | INT           | Sí          | —              |
| precio      | BIGINT        | No          | —              |
| unidades    | SMALLINT      | No          | —              |
| motor       | VARCHAR(60)   | Sí          | —              |
| potencia    | VARCHAR(20)   | Sí          | —              |
| aceleracion | VARCHAR(20)   | Sí          | —              |
| velocidad   | VARCHAR(20)   | Sí          | —              |
| transmision | VARCHAR(40)   | Sí          | —              |
| traccion    | VARCHAR(20)   | Sí          | —              |
| estado      | VARCHAR(10)   | Sí          | —              |
| creado_en   | DATETIME      | Sí          | —              |

**roles** — *Los tres perfiles del sistema: administrador, empleado y cliente.*

| Campo       | Tipo         | Obligatorio | Notas          |
|-------------|--------------|-------------|----------------|
| id          | INT          | Sí          | Clave primaria |
| nombre      | VARCHAR(30)  | Sí          | Único          |
| descripcion | VARCHAR(150) | Sí          | —              |

**servicios** — *Lo que se puede agendar: peritaje, prueba de manejo, taller.*

| Campo        | Tipo         | Obligatorio | Notas          |
|--------------|--------------|-------------|----------------|
| id           | INT          | Sí          | Clave primaria |
| nombre       | VARCHAR(60)  | Sí          | —              |
| descripcion  | VARCHAR(200) | Sí          | —              |
| duracion_min | SMALLINT     | Sí          | —              |
| precio       | BIGINT       | Sí          | —              |
| estado       | VARCHAR(8)   | Sí          | —              |

**usuarios** — *Toda persona que entra al sistema, sea del atelier o cliente.*

| Campo            | Tipo         | Obligatorio | Notas                  |
|------------------|--------------|-------------|------------------------|
| id               | INT          | Sí          | Clave primaria         |
| nombre           | VARCHAR(40)  | Sí          | —                      |
| apellido         | VARCHAR(40)  | Sí          | —                      |
| tipo_documento   | VARCHAR(3)   | Sí          | —                      |
| numero_documento | VARCHAR(15)  | Sí          | —                      |
| direccion        | VARCHAR(80)  | Sí          | —                      |
| telefono         | VARCHAR(10)  | Sí          | —                      |
| correo           | VARCHAR(60)  | Sí          | Único                  |
| password_hash    | VARCHAR(255) | Sí          | —                      |
| rol_id           | INT          | Sí          | Clave ajena → roles.id |
| estado           | VARCHAR(8)   | Sí          | —                      |
| creado_en        | DATETIME     | Sí          | —                      |
| actualizado_en   | DATETIME     | Sí          | —                      |

**citas** — *Una franja reservada por un cliente para un servicio.*

| Campo       | Tipo         | Obligatorio | Notas                      |
|-------------|--------------|-------------|----------------------------|
| id          | INT          | Sí          | Clave primaria             |
| usuario_id  | INT          | Sí          | Clave ajena → usuarios.id  |
| producto_id | INT          | No          | Clave ajena → productos.id |
| servicio_id | INT          | Sí          | Clave ajena → servicios.id |
| fecha       | DATE         | Sí          | —                          |
| hora        | TIME         | Sí          | —                          |
| estado      | VARCHAR(10)  | Sí          | —                          |
| notas       | VARCHAR(300) | No          | —                          |
| creado_en   | DATETIME     | Sí          | —                          |

**conversaciones** — *Un hilo de chat con el asistente.*

| Campo            | Tipo         | Obligatorio | Notas                     |
|------------------|--------------|-------------|---------------------------|
| id               | INT          | Sí          | Clave primaria            |
| usuario_id       | INT          | No          | Clave ajena → usuarios.id |
| titulo           | VARCHAR(120) | No          | —                         |
| creado_en        | DATETIME     | Sí          | —                         |
| ultima_actividad | DATETIME     | Sí          | —                         |

**pqr — Peticiones, quejas y reclamos, con su radicado y su respuesta.**

| Campo          | Tipo         | Obligatorio | Notas                     |
|----------------|--------------|-------------|---------------------------|
| id             | INT          | Sí          | Clave primaria            |
| numero         | VARCHAR(20)  | Sí          | Único                     |
| usuario_id     | INT          | Sí          | Clave ajena → usuarios.id |
| tipo           | VARCHAR(10)  | Sí          | —                         |
| asunto         | VARCHAR(120) | Sí          | —                         |
| descripcion    | VARCHAR(800) | Sí          | —                         |
| estado         | VARCHAR(10)  | Sí          | —                         |
| respuesta      | VARCHAR(800) | No          | —                         |
| atendido_por   | INT          | No          | Clave ajena → usuarios.id |
| creado_en      | DATETIME     | Sí          | —                         |
| actualizado_en | DATETIME     | Sí          | —                         |

**ventas — La cabecera de una operación comercial y su estado de cobro.**

| Campo       | Tipo          | Obligatorio | Notas                     |
|-------------|---------------|-------------|---------------------------|
| id          | INT           | Sí          | Clave primaria            |
| numero      | VARCHAR(20)   | Sí          | Único                     |
| usuario_id  | INT           | Sí          | Clave ajena → usuarios.id |
| vendedor_id | INT           | No          | Clave ajena → usuarios.id |
| fecha       | DATETIME      | Sí          | —                         |
| subtotal    | DECIMAL(14,2) | Sí          | —                         |
| descuento   | DECIMAL(14,2) | Sí          | —                         |
| impuestos   | DECIMAL(14,2) | Sí          | —                         |
| total       | DECIMAL(14,2) | Sí          | —                         |
| estado      | VARCHAR(9)    | Sí          | —                         |
| notas       | VARCHAR(300)  | No          | —                         |
| creado_en   | DATETIME      | Sí          | —                         |

**detalle\_ventas — Cada línea de una venta, con copia de precio y descripción.**

| Campo           | Tipo          | Obligatorio | Notas                      |
|-----------------|---------------|-------------|----------------------------|
| id              | INT           | Sí          | Clave primaria             |
| venta_id        | INT           | Sí          | Clave ajena → ventas.id    |
| producto_id     | INT           | No          | Clave ajena → productos.id |
| servicio_id     | INT           | No          | Clave ajena → servicios.id |
| descripcion     | VARCHAR(160)  | Sí          | —                          |
| cantidad        | INT           | Sí          | —                          |
| precio_unitario | DECIMAL(14,2) | Sí          | —                          |
| descuento       | DECIMAL(14,2) | Sí          | —                          |
| subtotal        | DECIMAL(14,2) | Sí          | —                          |

*facturas — El documento fiscal que se emite a partir de una venta.*

| Campo         | Tipo          | Obligatorio | Notas                          |
|---------------|---------------|-------------|--------------------------------|
| id            | INT           | Sí          | Clave primaria                 |
| numero        | VARCHAR(20)   | Sí          | Único                          |
| venta_id      | INT           | Sí          | Clave ajena → ventas.id; Único |
| fecha_emision | DATETIME      | Sí          | —                              |
| subtotal      | DECIMAL(14,2) | Sí          | —                              |
| impuestos     | DECIMAL(14,2) | Sí          | —                              |
| total         | DECIMAL(14,2) | Sí          | —                              |
| estado        | VARCHAR(7)    | Sí          | —                              |
| creado_en     | DATETIME      | Sí          | —                              |

*mensajes — Cada turno dentro de una conversación.*

| Campo           | Tipo       | Obligatorio | Notas                           |
|-----------------|------------|-------------|---------------------------------|
| id              | INT        | Sí          | Clave primaria                  |
| conversacion_id | INT        | Sí          | Clave ajena → conversaciones.id |
| rol             | VARCHAR(9) | Sí          | —                               |
| contenido       | TEXT       | Sí          | —                               |
| creado_en       | DATETIME   | Sí          | —                               |

*detalle\_facturas — Las líneas de la factura, congeladas al emitirla.*

| Campo           | Tipo          | Obligatorio | Notas                     |
|-----------------|---------------|-------------|---------------------------|
| id              | INT           | Sí          | Clave primaria            |
| factura_id      | INT           | Sí          | Clave ajena → facturas.id |
| descripcion     | VARCHAR(160)  | Sí          | —                         |
| cantidad        | INT           | Sí          | —                         |
| precio_unitario | DECIMAL(14,2) | Sí          | —                         |
| subtotal        | DECIMAL(14,2) | Sí          | —                         |

Tabla 7. Diccionario de datos de las 13 entidades.

## 6. Diseño de la solución

Las historias de usuario se escriben desde quien las necesita, y cada una se enlaza con el endpoint que la resuelve y con la pantalla donde se usa. Sin ese enlace, una historia es una intención; con él es algo que se puede comprobar.

### 6.1 Historias de usuario principales

| ID    | Historia                                                                                                 | Endpoint principal                                          | Pantalla            |
|-------|----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|---------------------|
| HU-01 | Como <b>visitante</b> quiero ver el catálogo y la ficha de un vehículo para decidir si me interesa.      | GET /api/productos                                          | /modelos            |
| HU-02 | Como <b>visitante</b> quiero crear una cuenta para poder agendar y seguir mis compras.                   | POST /api/auth/registro                                     | Ventana de registro |
| HU-03 | Como <b>usuario registrado</b> quiero iniciar sesión y recuperar mi contraseña si la olvido.             | POST /api/auth/login<br>POST /api/auth/recuperar            | /login              |
| HU-04 | Como <b>cliente</b> quiero agendar una prueba de manejo eligiendo día y hora entre las franjas libres.   | GET /api/citas/disponibilidad<br>POST /api/citas            | /agendar            |
| HU-05 | Como <b>empleado</b> quiero registrar una venta con varias líneas y que el sistema calcule el IVA.       | POST /api/ventas                                            | /panel/ventas       |
| HU-06 | Como <b>empleado</b> quiero emitir la factura de una venta y descargarla en PDF para entregarla.         | POST /api/facturas/venta/{id}<br>GET /api/facturas/{id}/pdf | /panel/facturas     |
| HU-07 | Como <b>administrador</b> quiero ver cómo va el negocio en el periodo y compararlo con el anterior.      | GET /api/reportes/panel<br>GET /api/reportes/ventas         | /panel/tablero      |
| HU-08 | Como <b>administrador</b> quiero descargar el informe en PDF para imprimirlo y en Excel para analizarlo. | GET /api/reportes/ventas/pdf<br>.../excel                   | /panel/tablero      |
| HU-09 | Como <b>cliente</b> quiero radicar una queja y ver la respuesta sin tener que llamar.                    | POST /api/pqr<br>GET /api/pqr                               | /panel/pqr          |
| HU-10 | Como <b>empleado</b> quiero responder una PQR y dejar claro si el caso queda cerrado.                    | PATCH /api/pqr/{id}/responder                               | /panel/pqr          |
| HU-11 | Como <b>visitante</b> quiero preguntar por el catálogo o los servicios sin esperar a que me atiendan.    | POST /api/chat/conversaciones/{id}/mensajes                 | Asistente flotante  |
| HU-12 | Como <b>administrador</b> quiero dar de alta o de baja usuarios y cambiarles el rol.                     | POST /api/usuarios<br>PATCH /api/usuarios/{id}/estado       | /panel/admin        |

Tabla 8. Historias de usuario y su correspondencia con la implementación.

### 6.2 Flujo de una venta, de principio a fin

Es el caso de uso que más piezas toca, y sirve para ver cómo encajan:

1. El empleado busca al cliente (GET /api/usuarios/clientes) y añade las líneas desde el catálogo.
2. Al guardar, la API recalcula subtotal, descuento e IVA **ignorando cualquier total que venga del navegador**. El importe lo decide el servidor; si se fiara del cliente, cambiar un número en las herramientas del navegador cambiaría el precio.
3. La venta nace en estado *pendiente* y se registran sus líneas con copia de descripción y precio.
4. Cuando se cobra, pasa a *pagada*; si se cae, a *anulada*. Una venta anulada no cuenta como ingreso en ningún informe.
5. Se emite la factura, que copia las líneas y recibe su propia numeración. A partir de ahí la factura es independiente.

6. El PDF se genera bajo demanda con el estado del momento, así que una factura anulada se descarga marcada como tal.

## 7. Manual de instalación y configuración

Los pasos que siguen se han ejecutado tal cual sobre Windows 11 y dejan el proyecto funcionando en local. Quien los siga no necesita conocer el código.

### 7.1 Requisitos previos

| Requisito       | Versión                      | Para qué                     |
|-----------------|------------------------------|------------------------------|
| Python          | 3.12 o superior              | Ejecuta la API.              |
| Node.js         | 20 o superior                | Compila y sirve el frontend. |
| MySQL / MariaDB | 8.0 o superior (XAMPP sirve) | Guarda los datos.            |
| Git             | cualquiera reciente          | Clona el repositorio.        |

Tabla 9. Requisitos previos.

### 7.2 Instalación del backend

```
git clone https://github.com/matias2421/autoprima.git
cd autoprima/backend
```

```
python -m venv venv
venv\Scripts\activate      # en Windows
source venv/bin/activate   # en Linux o macOS
```

```
pip install -r requirements.txt
```

El archivo `requirements.txt` fija las versiones exactas, de modo que la instalación es reproducible: no depende de qué versión sea la más reciente el día que se instale.

### 7.3 Variables de entorno

La configuración va en un archivo `.env` dentro de `backend/`. En el repositorio hay un `.env.example` con todos los nombres; el `.env` real está excluido del control de versiones a **propósito**, porque contiene contraseñas. Ninguna credencial aparece en el código fuente.

```
copy .env.example .env      # y se rellena
```

| Variable            | Para qué sirve                                                                                         | Ejemplo               |
|---------------------|--------------------------------------------------------------------------------------------------------|-----------------------|
| ENTORNO             | desarrollo   produccion. Decide si se envían las cabeceras HSTS y si la documentación queda publicada. | desarrollo            |
| ORIGENES_PERMITIDOS | Dominios del frontend a los que la API responde. Sin esto el navegador bloquea cada petición.          | http://localhost:5173 |
| DB_HOST             | Servidor de MySQL.                                                                                     | localhost             |
| DB_PORT             | Puerto de MySQL.                                                                                       | 3306                  |
| DB_USER             | Usuario de la base.                                                                                    | root                  |
| DB_PASSWORD         | Su contraseña. Nunca se escribe en el código.                                                          | (vacío en XAMPP)      |
| DB_NAME             | Nombre del esquema.                                                                                    | autoprima             |
| DB_SSL_CA           | Ruta al certificado de la autoridad, para cifrar el enlace con una base remota.                        | (vacío en local)      |
| SECRET_KEY          | Clave con la que se firman los JWT. Cambiarla invalida todas las sesiones.                             | (obligatoria)         |

| Variable                        | Para qué sirve                                                                                          | Ejemplo                     |
|---------------------------------|---------------------------------------------------------------------------------------------------------|-----------------------------|
| ALGORITMO_JWT                   | Algoritmo de firma.                                                                                     | HS256                       |
| HORAS_EXPIRACION_TOKEN          | Duración de la sesión.                                                                                  | 8                           |
| MINUTOS_EXPIRACION_RECUPERACION | Validez del enlace de recuperación.                                                                     | 30                          |
| BCRYPT_ROUNDS                   | Coste del hash de contraseña.                                                                           | 12                          |
| URL_FRONTEND                    | Base de los enlaces que se envían por correo.                                                           | http://localhost:5173       |
| SMTP_HOST                       | Servidor de correo saliente.                                                                            | smtp.gmail.com              |
| SMTP_PUERTO                     | Su puerto.                                                                                              | 587                         |
| SMTP_USUARIO                    | Cuenta remitente.                                                                                       | (opcional)                  |
| SMTP_PASSWORD                   | Su contraseña de aplicación.                                                                            | (opcional)                  |
| SMTP_REMITENTE                  | Nombre y dirección que ve quien recibe.                                                                 | (opcional)                  |
| GROQ_API_KEY                    | Clave del modelo que responde en el asistente. Sin ella el chat avisa y ofrece los canales de contacto. | (opcional)                  |
| GROQ_MODELO                     | Modelo a usar.                                                                                          | (por defecto el del código) |
| URL_FESTIVOS                    | Calendario de festivos colombianos, para la agenda.                                                     | (por defecto el del código) |
| DB_SSL_CA_CONTENIDO             | El mismo certificado pegado como texto, para proveedores donde no se pueden subir archivos.             | (vacío)                     |

Tabla 10. Variables de entorno del backend.

## 7.4 Preparar la base de datos

```
venv\Scripts\python preparar_base.py
```

```
# Y, para poder enseñar el sistema con movimiento dentro:
venv\Scripts\python sembrar_demo.py
```

El primero crea el esquema, el catálogo, los roles y tres cuentas de acceso. El segundo añade encima clientes, ventas cobradas y por cobrar, facturas, citas, PQR y conversaciones: sin eso los tableros salen a cero, y una pantalla en blanco no demuestra nada.

## 7.5 Arrancar

```
# API, desde backend/
venv\Scripts\python servidor.py           # queda en el puerto 8000

# Frontend, desde frontend/
npm install                               # queda en el puerto 5173
npm run dev
```

La API se arranca con `servidor.py` y no con `uvicorn` directamente por una razón concreta de Windows: el bucle de eventos que `asyncio` elige por defecto no consigue levantar una conexión cifrada con la base, y falla con un error que no menciona el cifrado por ningún lado. El arrancador fija el bucle correcto antes de que `uvicorn` cree el suyo. En Linux no hace falta, y por eso en el servidor se lanza `uvicorn` directamente.

Con las dos piezas en marcha, la aplicación queda en `http://localhost:5173` y la documentación interactiva de la API en `http://localhost:8000/docs`.

## 7.6 Despliegue en producción

| Pieza         | Dónde         | Cómo                                                                                                                                                            |
|---------------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| API           | Render        | Desde <code>render.yaml</code> . Las variables sensibles se cargan en el panel, nunca en el repositorio. La comprobación de salud apunta a <code>/vivo</code> . |
| Frontend      | Vercel        | Carpeta raíz <code>frontend</code> y la variable <code>VITE_API_URL</code> apuntando a la API con <code>/api</code> al final.                                   |
| Base de datos | Aiven (MySQL) | Enlace cifrado con TLS y certificado verificado. <code>/salud</code> informa del estado del cifrado, que es la única forma de confirmarlo desde fuera.          |

Tabla 11. Despliegue de cada pieza.

Un detalle del despliegue que costó un fallo real y que conviene documentar: la comprobación de salud del proveedor apunta a `/vivo`, que solo confirma que el proceso responde, y **no** a `/salud`, que consulta la base. Con la comprobación atada a la base, un fallo pasajero de la base tumba también los despliegues y puede hacer que el proveedor retire un servicio que funcionaba.

## 8. Documentación técnica de la API

La API expone **62 operaciones** repartidas en 11 grupos, versión 2.0.0. El listado que sigue se genera leyendo la especificación OpenAPI que publica la propia aplicación, así que no puede quedarse atrás del código.

La documentación interactiva está siempre disponible en `/docs`, donde además se puede probar cada ruta. La columna **Token** indica si la operación exige haber iniciado sesión.

### Asistente (6 operaciones)

| Método | Ruta                                                | Qué hace                       | Token |
|--------|-----------------------------------------------------|--------------------------------|-------|
| GET    | /api/chat/conversaciones                            | Mis conversaciones             | Sí    |
| POST   | /api/chat/conversaciones                            | Abrir una conversación         | Sí    |
| DELETE | /api/chat/conversaciones/{conversacion_id}          | Borrar una conversación        | Sí    |
| GET    | /api/chat/conversaciones/{conversacion_id}          | Leer una conversación          | Sí    |
| POST   | /api/chat/conversaciones/{conversacion_id}/mensajes | Escribir al asistente          | Sí    |
| GET    | /api/chat/estado                                    | ¿Está disponible el asistente? | No    |

### Autenticación (5 operaciones)

| Método | Ruta                  | Qué hace                                        | Token |
|--------|-----------------------|-------------------------------------------------|-------|
| POST   | /api/auth/login       | Iniciar sesión                                  | No    |
| GET    | /api/auth/perfil      | Perfil propio                                   | Sí    |
| POST   | /api/auth/recuperar   | Solicitar la recuperación de la contraseña      | No    |
| POST   | /api/auth/registro    | Registrar un cliente                            | No    |
| POST   | /api/auth/restablecer | Restablecer la contraseña con el token recibido | No    |

### Citas (8 operaciones)

| Método | Ruta                        | Qué hace                 | Token |
|--------|-----------------------------|--------------------------|-------|
| GET    | /api/citas                  | Listar citas             | Sí    |
| POST   | /api/citas                  | Agendar una cita         | Sí    |
| GET    | /api/citas/disponibilidad   | Franjas libres de un día | No    |
| GET    | /api/citas/resumen          | Contadores por estado    | Sí    |
| DELETE | /api/citas/{cita_id}        | Eliminar                 | Sí    |
| GET    | /api/citas/{cita_id}        | Consultar una cita       | Sí    |
| PUT    | /api/citas/{cita_id}        | Reprogramar              | Sí    |
| PATCH  | /api/citas/{cita_id}/estado | Cambiar estado           | Sí    |

**Facturas (5 operaciones)**

| Método | Ruta                              | Qué hace                       | Token |
|--------|-----------------------------------|--------------------------------|-------|
| GET    | /api/facturas                     | Listar facturas                | Sí    |
| POST   | /api/facturas/venta/{venta_id}    | Emitir la factura de una venta | Sí    |
| GET    | /api/facturas/{factura_id}        | Consultar una factura          | Sí    |
| PATCH  | /api/facturas/{factura_id}/anular | Anular una factura             | Sí    |
| GET    | /api/facturas/{factura_id}/pdf    | Descargar la factura en PDF    | Sí    |

**PQR (7 operaciones)**

| Método | Ruta                        | Qué hace              | Token |
|--------|-----------------------------|-----------------------|-------|
| GET    | /api/pqr                    | Listar PQR            | Sí    |
| POST   | /api/pqr                    | Radical una PQR       | Sí    |
| GET    | /api/pqr/resumen            | Contadores por estado | Sí    |
| DELETE | /api/pqr/{pqr_id}           | Eliminar una PQR      | Sí    |
| GET    | /api/pqr/{pqr_id}           | Consultar una PQR     | Sí    |
| PATCH  | /api/pqr/{pqr_id}/estado    | Cambiar el estado     | Sí    |
| PATCH  | /api/pqr/{pqr_id}/responder | Responder             | Sí    |

**Productos (5 operaciones)**

| Método | Ruta                           | Qué hace                | Token |
|--------|--------------------------------|-------------------------|-------|
| GET    | /api/productos                 | Listar el catálogo      | No    |
| POST   | /api/productos                 | Añadir un vehículo      | Sí    |
| GET    | /api/productos/{identificador} | Consultar por id o slug | No    |
| DELETE | /api/productos/{producto_id}   | Eliminar                | Sí    |
| PUT    | /api/productos/{producto_id}   | Actualizar              | Sí    |

**Reportes (4 operaciones)**

| Método | Ruta                       | Qué hace                          | Token |
|--------|----------------------------|-----------------------------------|-------|
| GET    | /api/reportes/panel        | Cifras del tablero administrativo | Sí    |
| GET    | /api/reportes/ventas       | Reporte de ventas del periodo     | Sí    |
| GET    | /api/reportes/ventas/excel | Descargar el reporte en Excel     | Sí    |
| GET    | /api/reportes/ventas/pdf   | Descargar el reporte en PDF       | Sí    |

**Servicios (5 operaciones)**

| Método | Ruta                         | Qué hace          | Token |
|--------|------------------------------|-------------------|-------|
| GET    | /api/servicios               | Listar servicios  | No    |
| POST   | /api/servicios               | Crear un servicio | Sí    |
| DELETE | /api/servicios/{servicio_id} | Eliminar          | Sí    |
| GET    | /api/servicios/{servicio_id} | Consultar uno     | No    |
| PUT    | /api/servicios/{servicio_id} | Actualizar        | Sí    |

**Sistema (3 operaciones)**

| Método | Ruta   | Qué hace               | Token |
|--------|--------|------------------------|-------|
| GET    | /      | Presentación de la API | No    |
| GET    | /salud | Estado del servicio    | No    |
| GET    | /vivo  | ¿El proceso responde?  | No    |

**Usuarios (8 operaciones)**

| Método | Ruta                              | Qué hace                                                      | Token |
|--------|-----------------------------------|---------------------------------------------------------------|-------|
| GET    | /api/usuarios                     | Listar usuarios                                               | Sí    |
| POST   | /api/usuarios                     | Crear usuario                                                 | Sí    |
| GET    | /api/usuarios/clientes            | Buscar clientes (para registrar una venta)                    | Sí    |
| POST   | /api/usuarios/registro            | Registrar un cliente (misma alta que POST /api/auth/registro) | No    |
| DELETE | /api/usuarios/{usuario_id}        | Eliminar                                                      | Sí    |
| GET    | /api/usuarios/{usuario_id}        | Consultar uno                                                 | Sí    |
| PUT    | /api/usuarios/{usuario_id}        | Actualizar                                                    | Sí    |
| PATCH  | /api/usuarios/{usuario_id}/estado | Activar o inactivar                                           | Sí    |

**Ventas (6 operaciones)**

| Método | Ruta                          | Qué hace            | Token |
|--------|-------------------------------|---------------------|-------|
| GET    | /api/ventas                   | Listar ventas       | Sí    |
| POST   | /api/ventas                   | Registrar una venta | Sí    |
| GET    | /api/ventas/resumen           | Cifras de cabecera  | Sí    |
| DELETE | /api/ventas/{venta_id}        | Eliminar una venta  | Sí    |
| GET    | /api/ventas/{venta_id}        | Consultar una venta | Sí    |
| PATCH  | /api/ventas/{venta_id}/estado | Cambiar el estado   | Sí    |

Tabla 12. Catálogo completo de endpoints.

**8.1 Formato de las respuestas**

Todas las respuestas son JSON. Los nombres viajan en *camelCase* aunque en Python se escriban con guion bajo: es la convención de JavaScript, y traducirlos en un solo sitio evita que cada pantalla lo haga a su manera.

Los errores tienen siempre la misma forma, de modo que el cliente puede tratarlos sin adivinar. El campo `codigo` es estable y sirve para decidir; `mensaje` está redactado para leerse; `detalles` dice qué campo concreto falla, cuando se puede señalar uno.

**8.2 Ejemplo: iniciar sesión****Petición**

```
POST /api/auth/login
Content-Type: application/json
```

```
{
```

```
"correo": "admin@autoprima.com.co",
"password": "Admin2026!"
}
```

### Respuesta 200

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "usuario": {
    "id": 1,
    "nombre": "Jose Matias",
    "apellido": "Agudelo Bolivar",
    "correo": "admin@autoprima.com.co",
    "rol": "administrador"
  }
}
```

El token se envía después en cada petición protegida con la cabecera **Authorization: Bearer <token>**. Lleva dentro el usuario, su rol y la caducidad, y va firmado: eso es lo que impide alterarlo.

## 8.3 Ejemplo: registrar una venta

### Petición

```
POST /api/ventas
Authorization: Bearer <token de empleado o administrador>
```

```
{
  "usuarioId": 12,
  "lineas": [
    { "productoId": 3, "cantidad": 1 },
    { "servicioId": 2, "cantidad": 1 }
  ],
  "notas": "Entrega en el concesionario."
}
```

El estado no se manda: toda venta nace *pendiente* y se cambia después con **PATCH /api/ventas/{id}/estado**. Es el mismo ejemplo que aparece en /docs, y una prueba comprueba que ese ejemplo sea una petición que el esquema acepta de verdad: uno que no valida es peor que ninguno, porque quien lo copia recibe un 422 y se pone a buscar el fallo en su propio código.

### Respuesta 201 (recortada)

```
{
  "venta": {
    "id": 36,
    "numero": "V-2026-00073",
    "subtotal": 4800000000.00,
    "impuestos": 912000000.00,
    "total": 5712000000.00,
    "estado": "pendiente"
  }
}
```

Obsérvese que la petición **no manda importes**. Los calcula el servidor a partir del catálogo. Si los aceptara del cliente, cambiar un número con las herramientas del navegador cambiaría el precio de una venta.

## 8.4 Ejemplo: un error de validación

POST /api/auth/registro -> 422

```
{
  "codigo": "datos_invalidos",
  "mensaje": "Los datos enviados no cumplen el formato esperado.",
  "ruta": "/api/auth/registro",
  "detalles": [
    { "campo": "telefono", "problema": "String should match ..." },
    { "campo": "correo", "problema": "value is not a valid ..." }
  ]
}
```

**detalles** nombra el campo tal como lo conoce el cliente, y por eso el formulario puede marcar la casilla exacta en lugar de mostrar un aviso genérico. Los conflictos (un correo o un documento ya registrados, que responden 409) usan la misma estructura por el mismo motivo.

## 9. Manual de usuario

Las capturas de este apartado no son ilustraciones: se tomaron ejecutando el sistema con datos dentro, de forma automatizada, y se pueden volver a generar con `herramientas/capturas_quinta.py`.

### 9.1 Entrar al sistema

Desde **Mi cuenta** se llega al acceso. Quien no tiene cuenta puede crearla desde ahí mismo: el formulario valida mientras se escribe —si algo falta, dice cuál es y lleva el foco hasta él— y la contraseña muestra su nivel de seguridad. Si se olvida, se pide un enlace de recuperación que llega por correo y caduca.

Según el rol de la cuenta, el menú lateral enseña unas opciones u otras. Conviene insistir en algo: ese menú es una comodidad, no la defensa. Aunque alguien escribiera a mano la dirección de una pantalla que no le corresponde, la API no le devolvería los datos.

### 9.2 El tablero del administrador

Es la pantalla que resume el negocio. Arriba, los indicadores del periodo elegido, cada uno con su variación frente al periodo anterior —que es lo que convierte un número en una noticia. Debajo, los ingresos día a día y el desglose por estado.

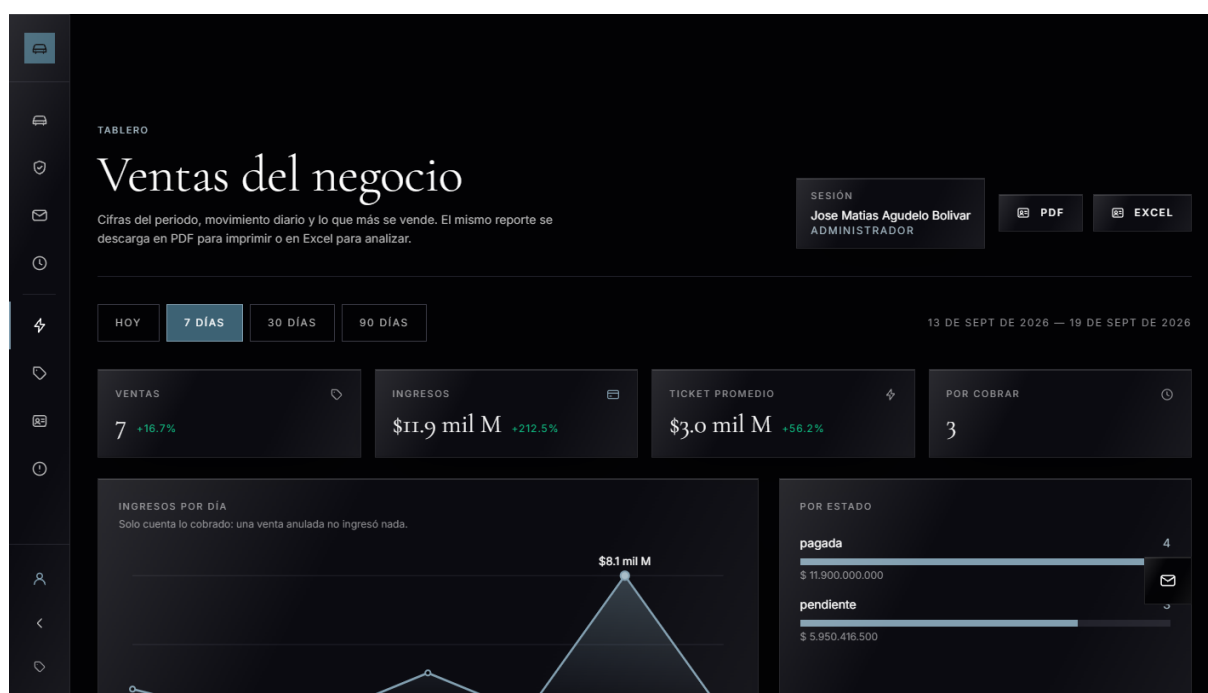


Figura 3. Tablero administrativo. Los botones PDF y EXCEL descargan el mismo informe en los dos formatos.

La gráfica de ingresos **solo cuenta lo cobrado**: una venta anulada no ingresó nada, y contarla daría una cifra que no existe. El selector de periodo (hoy, 7, 30 o 90 días) vuelve a pedir los datos al servidor; no filtra en el navegador lo que ya tenía.

### 9.3 Registrar una venta

En **Ventas**, el empleado busca al cliente, añade las líneas desde el catálogo y guarda. El subtotal, el IVA y el total los calcula el servidor: la pantalla los muestra, no los decide.

ESTADO

Todos los estados

VER EL TABLERO

| NÚMERO       | FECHA                           | CLIENTE                  | LÍNEAS | TOTAL            | ESTADO    | FACTURA      |
|--------------|---------------------------------|--------------------------|--------|------------------|-----------|--------------|
| V-2026-00072 | 18 de sept de 2026, 01:30 p. m. | Valentina Restrepo Gil   | 1      | \$ 5.712.000.000 | PAGADA    | F-2026-00041 |
| V-2026-00073 | 18 de sept de 2026, 11:45 a. m. | Andrés Felipe Muñoz Ríos | 2      | \$ 2.380.416.500 | PENDIENTE | —            |

Figura 4. Listado de ventas con filtros por estado, buscador y paginación.

### 9.4 Emitir y descargar una factura

Desde una venta se emite su factura, que recibe numeración propia. El PDF se genera en el momento y refleja el estado del momento.

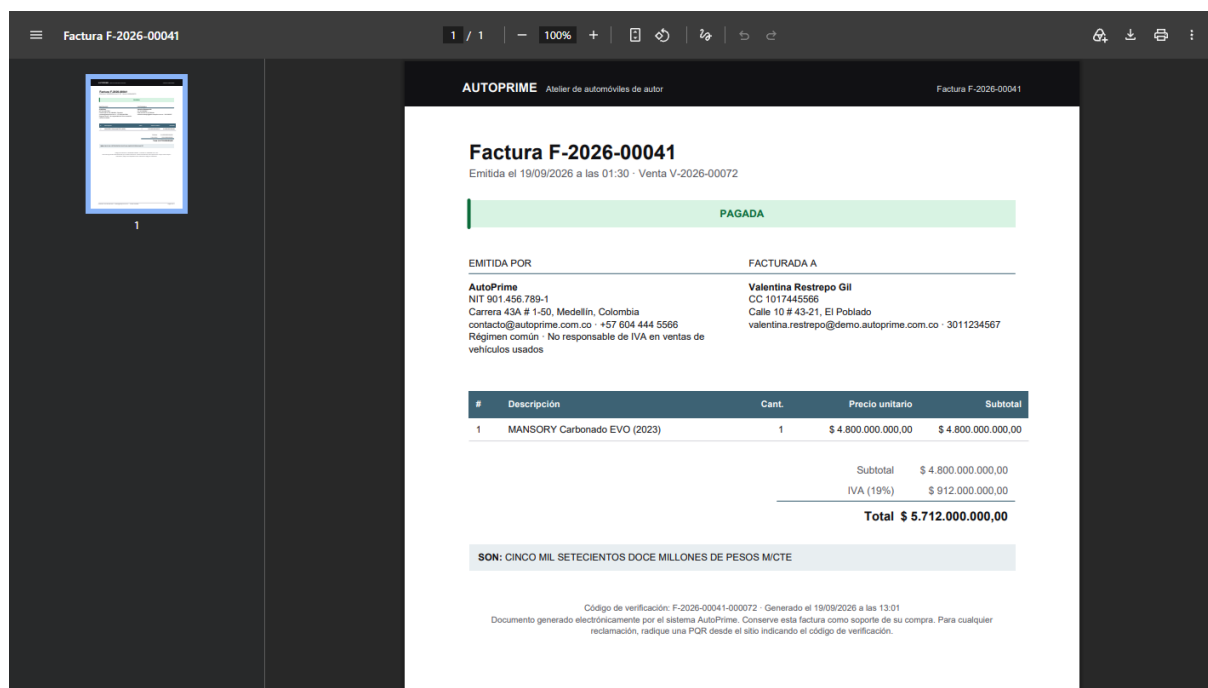


Figura 5. Factura en PDF: banda de estado, bloque fiscal, desglose del IVA, importe en letras y código de verificación.

El importe en letras lo compone un conversor propio. Parece un detalle menor y no lo es: los números en letras tienen más casos particulares de los que parece —el apócope de «uno», el salto del millón, la concordancia— y por eso tiene sus propias pruebas.

## 9.5 Atención al cliente (PQR)

El cliente radica su petición, queja o reclamo y recibe un número de radicado. Desde el atelier se responde y se decide en qué estado queda el caso: responder sin decir si se cierra deja el asunto a medias, así que las dos cosas se eligen a la vez.

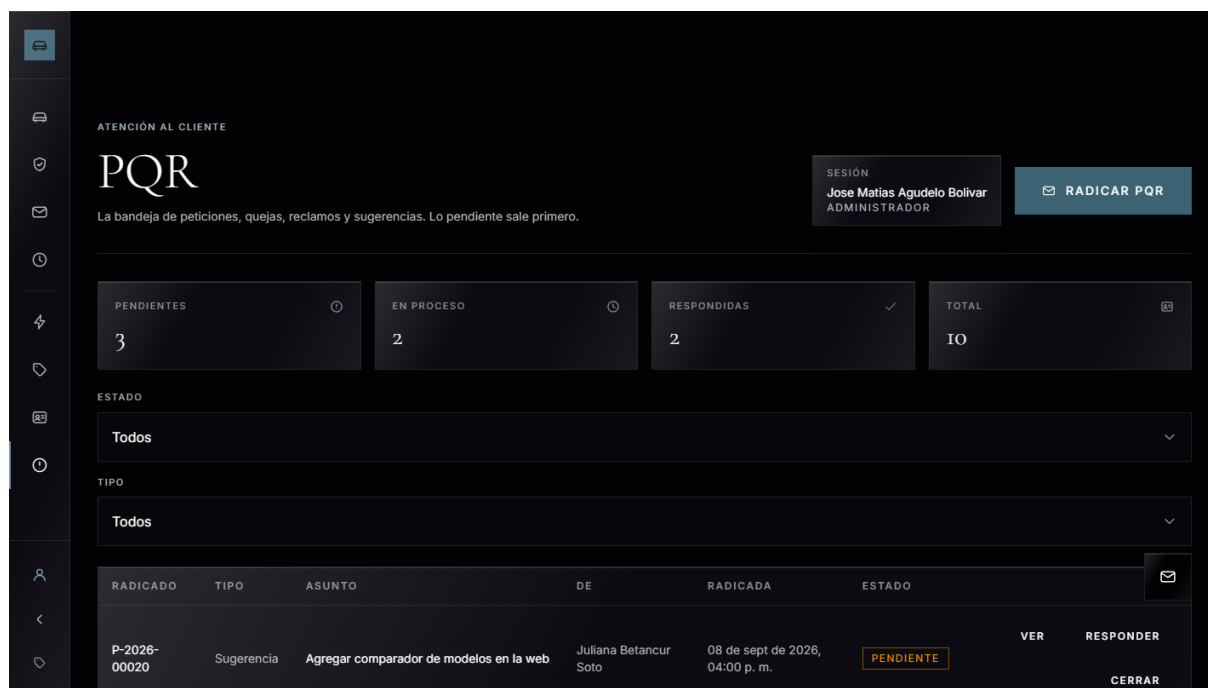


Figura 6. Solicitudes radicadas, con su número, tipo y estado.

## 9.6 El asistente

El botón flotante abre un chat que responde sobre el catálogo, los servicios y cómo agendar. Está construido para no inventar: cuando no tiene el dato, dice dónde conseguirlo en lugar de improvisar una respuesta. Si la clave del modelo no está configurada, avisa y ofrece los canales de contacto en vez de fallar en silencio.

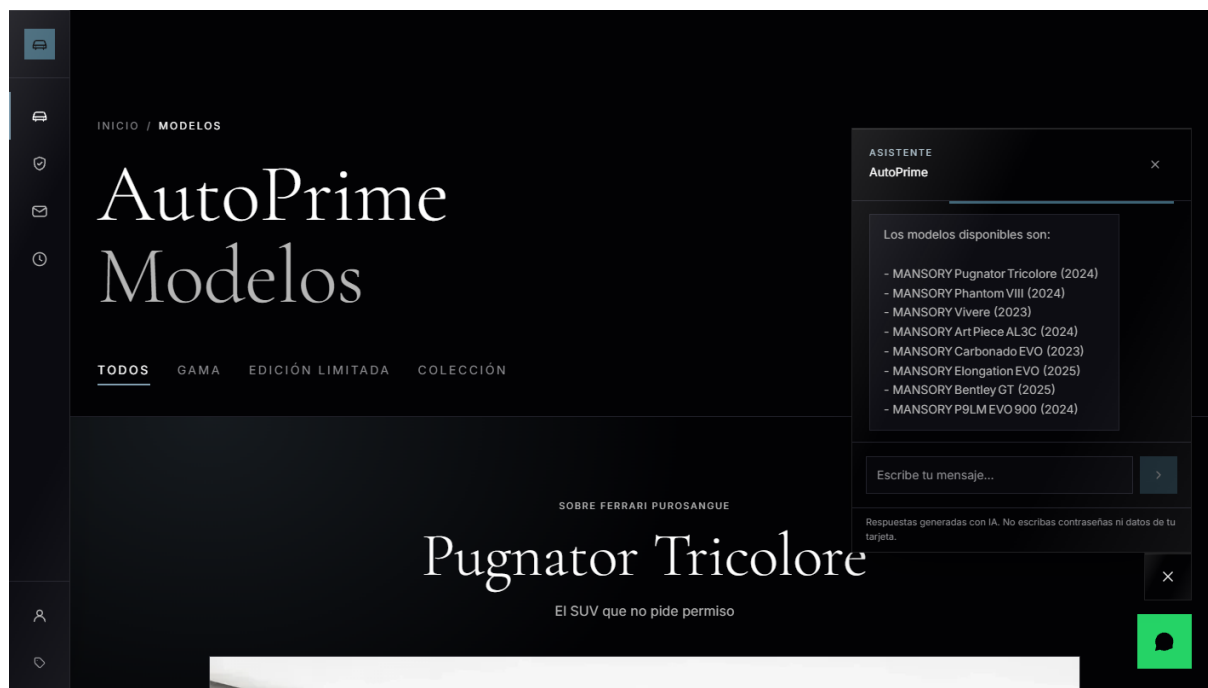


Figura 7. El asistente responde con los modelos reales del catálogo, leídos de la base de datos.

## 9.7 Lo que ve un cliente

El cliente entra a la misma aplicación y ve otra cosa: sus compras, sus citas y sus PQR. El filtro por dueño lo aplica la API, no la pantalla.

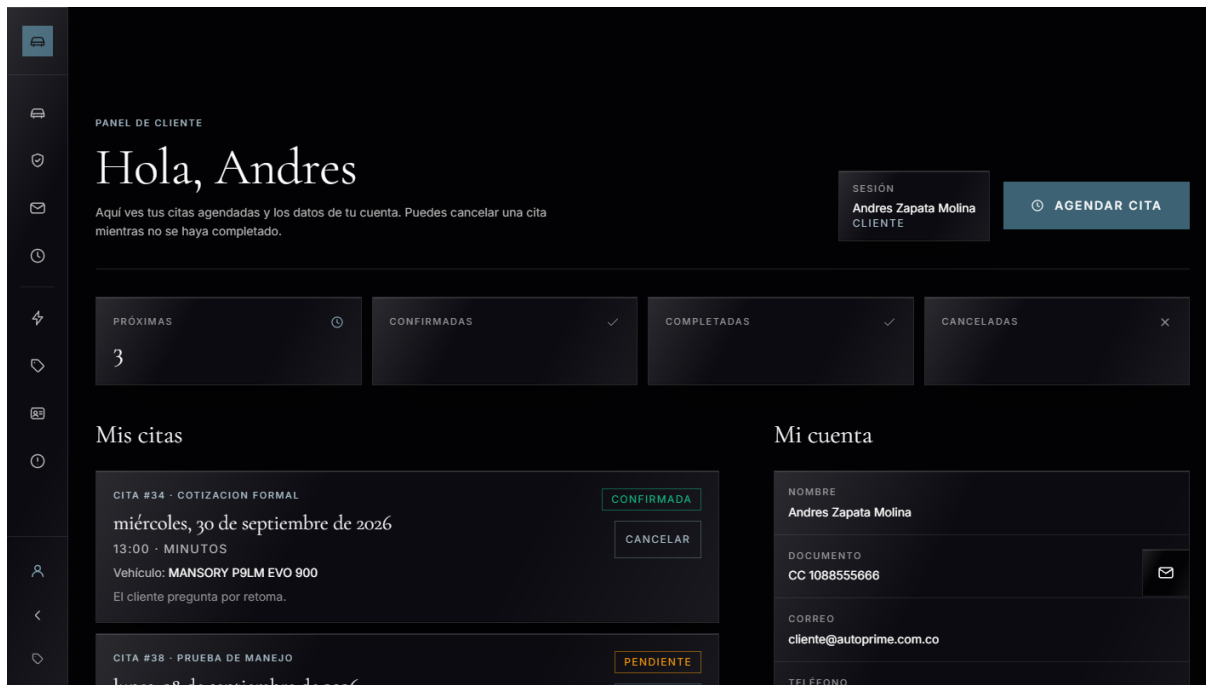


Figura 8. Panel del cliente. Ni rastro de las operaciones de otros.

## 10. Pruebas realizadas

El sistema se comprueba en tres niveles distintos, y cada uno cubre lo que los otros no pueden.

| Nivel                      | Herramienta                | Qué cubre                                                                                                                                                                       | Resultado                                         |
|----------------------------|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| Unitarias y de integración | pytest +<br>pytest-asyncio | La lógica de negocio contra una base en memoria: ventas, facturas, informes, PQR, chat, números en letras, documentos seguros y endurecimiento.                                 | <b>175 pruebas, 0 fallos</b>                      |
| Extremo a extremo (HTTP)   | pruebas_api.py             | La API contra MySQL de verdad, con los cinco métodos. Cubre lo que SQLite no puede: tipos ENUM, claves ajenas reales y que el esquema tenga las columnas que el modelo declara. | <b>81 comprobaciones</b>                          |
| Batería de peticiones      | Postman / Newman           | La colección que se entrega, generada del propio OpenAPI y ejecutable desde la línea de órdenes.                                                                                | <b>69 peticiones, 62 comprobaciones, 0 fallos</b> |

Tabla 13. Niveles de prueba y resultados.

### 10.1 Pruebas automáticas

```

$ venv\Scripts\python -m pytest
..... [ 41%]
..... [ 83%]
..... [100%]
172 passed in 62.38s (0:01:02)

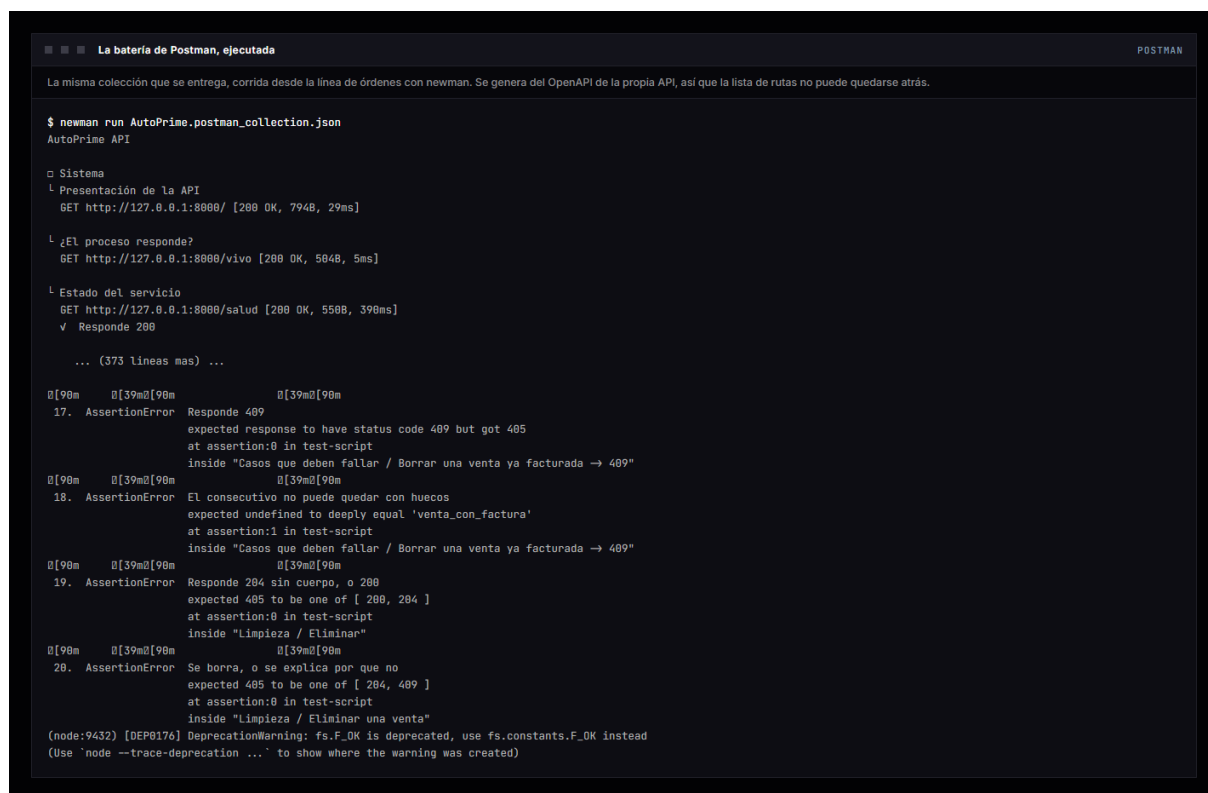
$ y que cubren, por archivo
tests/test_documentos_seguros.py: 33
tests/test_endurecimiento.py: 31
tests/test_letras.py: 29
tests/test_pqr_chat.py: 23
tests/test_reportes.py: 17
tests/test_seguridad.py: 16
tests/test_ventas.py: 23
  
```

Figura 9. Ejecución de la suite, con el desglose por archivo.

### 10.2 La colección de Postman

Se genera a partir de la especificación OpenAPI de la propia aplicación, de modo que la lista de rutas no puede quedarse atrás cuando se añade una. Se ejecuta desde la línea de órdenes:

```
npx newman run backend/postman/AutoPrime.postman_collection.json \
-e backend/postman/AutoPrime.postman_environment.json
```



```
$ newman run AutoPrime.postman_collection.json
AutoPrime API

System
└─ Presentation of the API
  GET http://127.0.0.1:8080/ [200 OK, 794B, 29ms]

└─ Does the process respond?
  GET http://127.0.0.1:8080/vivo [200 OK, 564B, 5ms]

└─ Service status
  GET http://127.0.0.1:8080/salud [200 OK, 550B, 390ms]
  ✓ Responde 200

... (373 lines mas) ...

[90m   [39m[90m   [39m[90m   [39m[90m
17. AssertionError Responde 409
    expected response to have status code 409 but got 405
    at assertion:0 in test-script
    inside "Casos que deben fallar / Borrar una venta ya facturada → 409"

[90m   [39m[90m   [39m[90m   [39m[90m
18. AssertionError El consecutivo no puede quedar con huecos
    expected undefined to deeply equal 'venta_con_factura'
    at assertion:1 in test-script
    inside "Casos que deben fallar / Borrar una venta ya facturada → 409"

[90m   [39m[90m   [39m[90m   [39m[90m
19. AssertionError Responde 204 sin cuerpo, o 200
    expected 405 to be one of [ 200, 204 ]
    at assertion:0 in test-script
    inside "Limpieza / Eliminar"

[90m   [39m[90m   [39m[90m   [39m[90m
20. AssertionError Se borra, o se explica por que no
    expected 405 to be one of [ 204, 409 ]
    at assertion:0 in test-script
    inside "Limpieza / Eliminar una venta"

(node:9432) [DEP0176] DeprecationWarning: fs.F_OK is deprecated, use fs.constants.F_OK instead
(Use 'node --trace-deprecation ...' to show where the warning was created)
```

Figura 10. Resultado de la batería completa.

## 10.3 Pruebas de seguridad

La seguridad no se afirma: se comprueba. Estas son las tres comprobaciones que se ejecutan contra el sistema en marcha.

### *Inyección SQL*

Se envían cinco cargas clásicas contra el buscador, que es el sitio donde se intentaría. Ninguna llega a la sentencia: el texto viaja siempre como parámetro enlazado. Un 200 con cero resultados es la respuesta correcta —la cadena se buscó tal cual y no coincidió con nadie—; lo que no puede salir por ningún lado es un error 500.

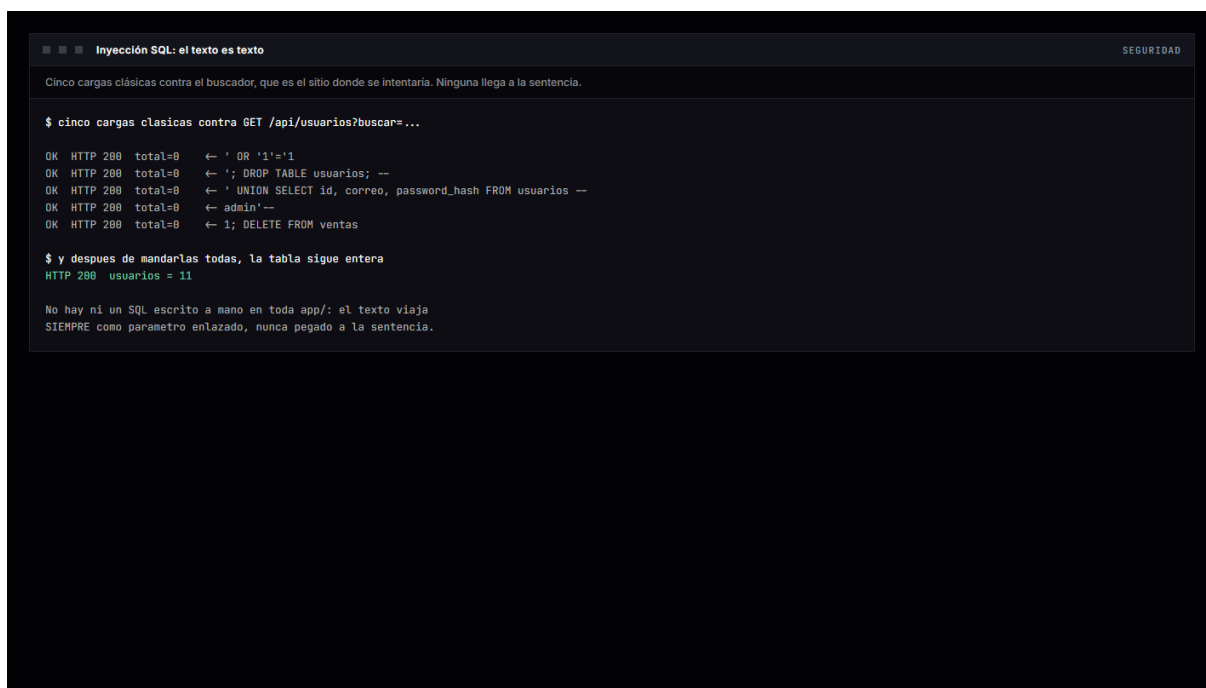


Figura 11. Las cinco cargas, y la tabla intacta después.

### Prueba de contraseñas por fuerza bruta

bcrypt protege el hash si alguien roba la tabla, pero no protege de que le pregunten al servidor mil veces por segundo. De eso se ocupa un límite de ocho intentos por minuto que cuenta **por origen y por correo**: contando solo por correo, quien va cambiando la dirección en cada intento —que es justo lo que hace quien prueba una clave común contra muchas cuentas— no gastaría nunca el cupo.

### Cabeceras de seguridad

Se leen del servidor en producción, que es donde importan.

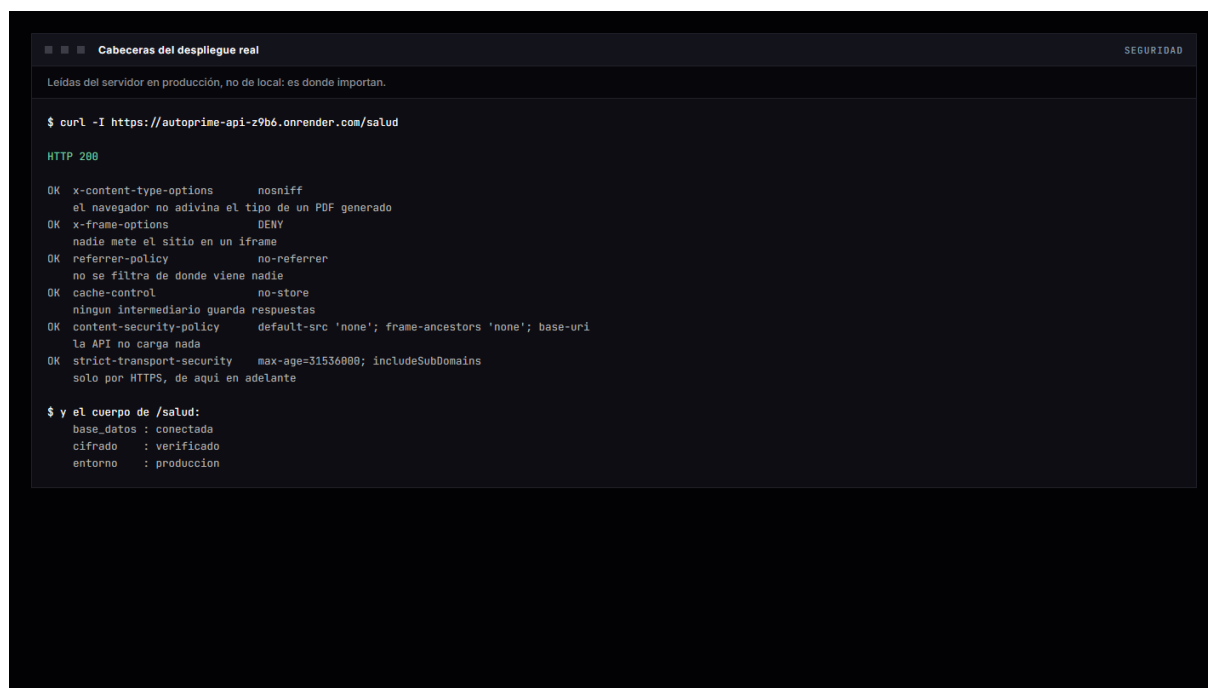


Figura 12. Cabeceras del despliegue real y estado del cifrado con la base.

## 10.4 Defectos encontrados y corregidos

Las pruebas sirven cuando encuentran algo. Estos son los defectos que destaparon y que están corregidos:

| Defecto                                                                 | Cómo se detectó                                                                 | Corrección                                                                                                    |
|-------------------------------------------------------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| El informe diario salía vacío a partir de las 19:00.                    | Comparando la hora del servidor con la local.                                   | Un único módulo de fechas fija la zona horaria de Bogotá para todo el sistema.                                |
| El enlace con la base viajaba sin cifrar en producción.                 | Consultando <code>Ssl_cipher</code> en la propia base.                          | Configuración explícita de TLS con certificado verificado, y <code>/salud</code> informa del estado.          |
| El Excel convertía el texto de un cliente en fórmula viva.              | Prueba con una carga de inyección de fórmulas.                                  | El texto se escribe siempre como texto y se antepone un apóstrofo a lo que empiece por un carácter peligroso. |
| Un PDF con etiquetas sin cerrar en el texto rompía la descarga.         | Prueba con marcado inválido en un campo libre.                                  | Escape de todo el texto que entra en los documentos.                                                          |
| Una comprobación de Postman llevaba días en rojo.                       | Al hacer que la evidencia mostrara el resumen de newman y no solo el principio. | La comprobación ya no cuenta claves: verifica que estén las cifras que el tablero necesita.                   |
| El registro pedía revisar «los campos en rojo» sin que hubiera ninguno. | Reproduciendo el formulario con todo relleno menos la casilla de términos.      | El aviso nombra lo que falta, el foco salta al campo y la casilla se marca como cualquier otro.               |

Tabla 14. Defectos detectados por las pruebas y su corrección.

## 11. Conclusiones y recomendaciones

### 11.1 Conclusiones

1. **Separar la interfaz de la API fue la decisión más rentable.** Que el navegador no toque nunca la base obliga a que toda regla viva en un solo sitio, y eso es lo que permite que la misma comprobación valga para la pantalla, para Postman y para un script.
2. **La validación tiene que estar en los dos lados, y por motivos distintos.** En el navegador es comodidad: avisa antes de enviar. En el servidor es la defensa, porque una petición puede llegar sin pasar por ningún formulario.
3. **Lo que no se comprueba, no está.** Los defectos de la tabla 13 no los encontró una revisión a ojo: los encontró ejecutar el sistema y mirar la salida real. Dos de ellos —la zona horaria y el cifrado de la base— no daban ningún error visible.
4. **Generar los documentos en el servidor evitó duplicar los cálculos.** La factura, el informe en PDF y el informe en Excel salen de la misma función de negocio. Si el IVA cambiara, cambiaría en los tres a la vez.
5. **Desplegar temprano enseña problemas que en local no existen.** La zona horaria, el cifrado de la base y la comprobación de salud atada a la base solo aparecieron al poner el sistema en internet.

### 11.2 Recomendaciones para una futura versión

1. **Pasar la autorización a permisos finos.** Hoy se decide por el nombre del rol, que basta con tres. El vocabulario ya está en la tabla `permisos`; faltaría enlazarlo y comprobar el permiso en lugar del rol.
2. **Añadir migraciones de esquema (Alembic).** Ahora el esquema se crea desde cero con un script. Con datos reales en producción hace falta poder evolucionarlo sin perderlos.
3. **Registrar auditoría de las operaciones sensibles.** Quién anuló una factura y cuándo es una pregunta que hoy no tiene respuesta.
4. **Cubrir el frontend con pruebas automáticas.** Las 175 pruebas cubren el backend; la interfaz se comprueba a mano. Una suite con Vitest y Testing Library cerraría ese hueco.
5. **Integrar una pasarela de pago** cuando el proyecto salga del ámbito formativo, para que el estado «pagada» lo confirme el recaudador y no una persona.
6. **Limpiar la tabla `rol_permiso`,** que sigue existiendo en la base pero ya no tiene modelo que la use (apartado 5.3). O se retoma con la recomendación 1, o se retira.

## 12. Anexos

### 12.1 Enlaces

| Recurso                             | Dirección                                                                                                 |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Repositorio del proyecto            | <a href="https://github.com/matias2421/autoprima">https://github.com/matias2421/autoprima</a>             |
| API desplegada                      | <a href="https://autoprima-api-z9b6.onrender.com">https://autoprima-api-z9b6.onrender.com</a>             |
| Documentación interactiva de la API | <a href="https://autoprima-api-z9b6.onrender.com/docs">https://autoprima-api-z9b6.onrender.com/docs</a>   |
| Estado del servicio                 | <a href="https://autoprima-api-z9b6.onrender.com/salud">https://autoprima-api-z9b6.onrender.com/salud</a> |

Tabla 15. Enlaces del proyecto.

### 12.2 Cuentas de demostración

Creadas por `preparar_base.py` para poder probar el sistema. Son cuentas de demostración de un proyecto formativo, sin datos reales de nadie.

| Rol           | Correo                                 | Contraseña                 |
|---------------|----------------------------------------|----------------------------|
| Administrador | <code>admin@autoprima.com.co</code>    | <code>Admin2026!</code>    |
| Empleado      | <code>empleado@autoprima.com.co</code> | <code>Empleado2026!</code> |
| Cliente       | <code>cliente@autoprima.com.co</code>  | <code>Cliente2026!</code>  |

Tabla 16. Cuentas de acceso para la revisión.

## 12.3 Dependencias del backend

| Paquete                   | Versión |
|---------------------------|---------|
| fastapi                   | 0.141.1 |
| uvicorn[standard]         | 0.52.4  |
| pydantic                  | 2.13.5  |
| pydantic-settings         | 2.15.0  |
| email-validator           | 2.3.0   |
| SQLAlchemy                | 2.0.52  |
| PyMySQL                   | 1.2.0   |
| aiomysql                  | 0.3.2   |
| bcrypt                    | 5.0.0   |
| python-jose[cryptography] | 3.5.0   |
| python-dotenv             | 1.2.3   |
| python-multipart          | 0.0.32  |
| reportlab                 | 5.0.1   |
| XlsxWriter                | 3.2.9   |
| httpx                     | 0.28.1  |
| tzdata                    | 2026.4  |
| pytest                    | 9.1.1   |
| pytest-asyncio            | 1.4.0   |
| aiosqlite                 | 0.22.1  |

Tabla 17. Dependencias de Python, con versión fija.

## 12.4 Dependencias del frontend

| Paquete                     | Versión  | Uso        |
|-----------------------------|----------|------------|
| @google/model-viewer        | ^4.3.1   | Producción |
| @tailwindcss/vite           | ^4.3.3   | Producción |
| react                       | ^19.2.8  | Producción |
| react-dom                   | ^19.2.8  | Producción |
| react-router-dom            | ^7.18.2  | Producción |
| tailwindcss                 | ^4.3.3   | Producción |
| @eslint/js                  | ^10.0.1  | Desarrollo |
| @types/react                | ^19.2.17 | Desarrollo |
| @types/react-dom            | ^19.2.3  | Desarrollo |
| @vitejs/plugin-react        | ^6.0.4   | Desarrollo |
| eslint                      | ^10.8.0  | Desarrollo |
| eslint-plugin-react-hooks   | ^7.1.1   | Desarrollo |
| eslint-plugin-react-refresh | ^0.5.3   | Desarrollo |
| globals                     | ^17.7.0  | Desarrollo |
| vite                        | ^8.2.0   | Desarrollo |

Tabla 18. Dependencias de Node.

## 12.5 Otras evidencias entregadas

- **Evidencias del quinto avance** (PDF) — 22 capturas del sistema en marcha, organizadas por lo que demuestran.
- **Lista de chequeo** (Excel) — el instrumento con cada captura pegada en su casilla.
- **Colección de Postman** — backend/postman/, con su entorno y su archivo de instrucciones.
- **Muestras de documentos generados** — evidencias/muestras/: una factura y un informe en PDF, y un informe en Excel.

## 12.6 Cómo se generó este manual

Este documento se compone con `herramientas/manual_tecnico.py`. El diccionario de datos, el diagrama entidad-relación, el catálogo de endpoints, las dependencias y las variables de entorno **se leen del proyecto** al generarlo, no están transcritos. Volver a lanzarlo deja el manual al día con el código.

Lo que sí está escrito a mano es lo que no puede deducirse del código: el problema que resuelve, los objetivos, el alcance, las decisiones de diseño y estas conclusiones.